
Oliver Krauss

Multithreading

ALD | MTD.ba VZ SS26

Hagenberg TBD

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Recap: Lecture 12 -- Design Patterns

Last session: reusable solutions to recurring design problems

Design patterns give a shared vocabulary for proven object-oriented solutions. Every pattern you used this semester now has a name.

Recap: Lecture 12 -- Design Patterns

Last session: reusable solutions to recurring design problems

Design patterns give a shared vocabulary for proven object-oriented solutions. Every pattern you used this semester now has a name.

Patterns covered:

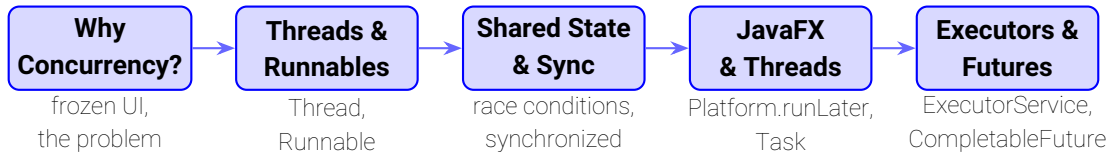
- **Creational:** Singleton, Factory Method, Builder
- **Structural:** Facade, Decorator, Proxy, Composite, Adapter
- **Behavioral:** Strategy, Command, Observer, State, Template Method, Memento, Chain of Resp., Visitor, Mediator

Principles:

- SOLID -- design intent behind every pattern
- YAGNI -- don't introduce a pattern before you have two concrete cases
- Anti-patterns: God Class, Singleton Abuse, Copy-Paste Coding

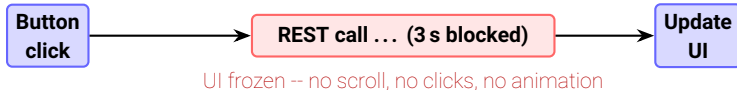
Today: Patterns tell you *how to structure* code. But what happens when your program needs to do more than one thing *at the same time*?

Today's Journey

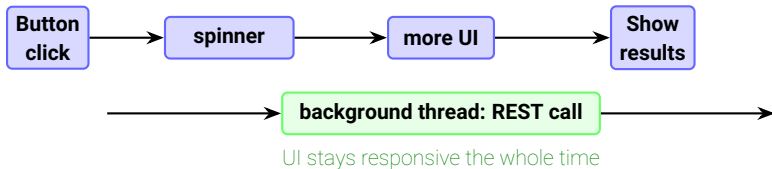


The Frozen UI Problem

Single thread:



With background thread:



One thread = one thing at a time. A blocking REST call or database query stops every UI event.

Concurrency vs. Parallelism

Concurrency -- managing multiple tasks

Tasks are in progress at the same time, but may not *execute* simultaneously. On a single core, the OS rapidly switches between them -- the illusion of simultaneous progress.

Example: a JavaFX app handling UI events and a loading spinner while waiting for network data -- even on one core.

Parallelism -- truly simultaneous execution

Multiple tasks literally run at the same instant, on separate CPU cores.

Example: a photo-processing app splitting an image into 8 tiles, each processed by a separate core, then merged.

Java's Thread Model Gives You Both

Concurrency and parallelism in practice

Java threads are mapped to OS threads. On modern multi-core CPUs they run in parallel. But even on a hypothetical single core, concurrency still prevents a blocked thread from freezing the whole program.

For this course, the goal is concurrency: keep the FX Application Thread free so the UI stays alive.

The Thread Model in Java

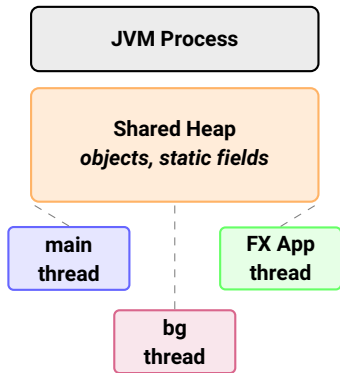
Every Java program starts with one thread:

the *main thread*, which runs `main()`.

A JavaFX app adds a second: the *FX Application Thread*, which owns all UI components. `Application.launch()` hands control to it.

Your code creates more -- background workers, timers, network listeners.

All of them live inside one **JVM process** and can access the same objects on the shared heap.



Shared Heap, Private Stacks

All threads share the same heap

Objects live on the heap and are visible to every thread in the JVM -- this is both powerful and dangerous. Threads have their own **stack** (local variables), but share all **fields** and **static** data.

Restaurant analogy

The FX thread is the waiter -- it must stay free to take orders. Background threads are the kitchen -- slow work happens there. **Platform.runLater()** is the pass-through window.

Creating a Thread -- Two Ways

Option A: extend Thread

```
1  public class Worker extends Thread {
2      @Override
3      public void run() {
4          System.out.println("Worker: "
5              + Thread.currentThread().
6                  getName());
7      }
8
9      // Start it:
10     new Worker().start();
```

Limitation: Java has single inheritance -- extending Thread locks you out of extending anything else.

Option B: Runnable (preferred)

```
1  Runnable task = () -> System.out.
    println("Worker: "
2      + Thread.currentThread().getName());
3
4  new Thread(task).start();
5
6  // Shorter with a lambda:
7  new Thread(() ->
8      System.out.println("Hello from bg!"
9          )
10     ).start();
```

***Runnable** is a functional interface (since Java 8). Lambdas and method references work directly.*

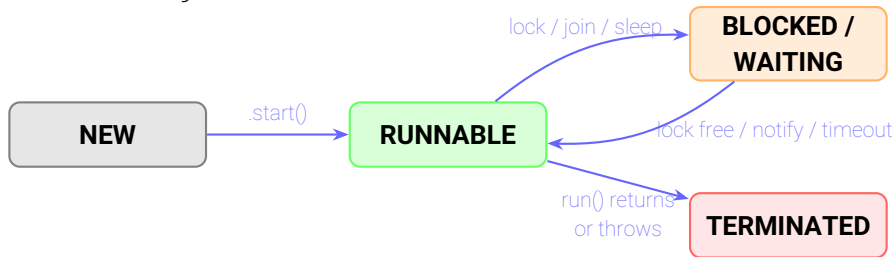
`.start()` vs `.run()` -- a Very Common Mistake

Never call `run()` when you mean `start()`

`thread.start()` asks the OS to create a new thread and calls `run()` on it.

`thread.run()` just calls the method **on the current thread** -- no new thread is created.

Thread Lifecycle



Paused states

A thread is paused -- waiting for a lock, a signal, or a timeout. It returns to **RUNNABLE** automatically when the condition is met.

TIMED_WAITING: used when `sleep(ms)` is called -- the JVM wakes the thread after the timeout even if nothing else signals it.

TERMINATED is final

Once a thread reaches **TERMINATED** it cannot be restarted. Create a new **Thread** instance if you need to run the same task again.

Key Thread Methods: `sleep` & `join`

```
1  Thread worker = new Thread(() -> {
2      System.out.println("Working...");
3      try {
4          Thread.sleep(2000); // pause 2s
5      } catch (InterruptedException e) {
6          Thread.currentThread().interrupt();
7          // restore flag!
8      }
9      System.out.println("Done.");
10 }
11 worker.start();
12 worker.join(); // main thread waits
13                here
14 System.out.println("Worker finished.");
```

`sleep(ms)`

Pauses the *current* thread for at least the given time. Does **not** release held locks. Throws **InterruptedException** if another thread calls `interrupt()` on it.

`join()`

The calling thread blocks until the target thread terminates. Useful to wait for a worker to finish before reading its result.

Handling `InterruptedException`

Never swallow it

Catching `InterruptedException` and doing nothing hides cancellation requests. Always either rethrow it or call `Thread.currentThread().interrupt()` to restore the flag.

Debugging tip

`Thread.currentThread().getName()` tells you which thread is running the current line. Print it inside callbacks and lambdas to verify your threading assumptions.

The Race Condition

Counter that loses increments

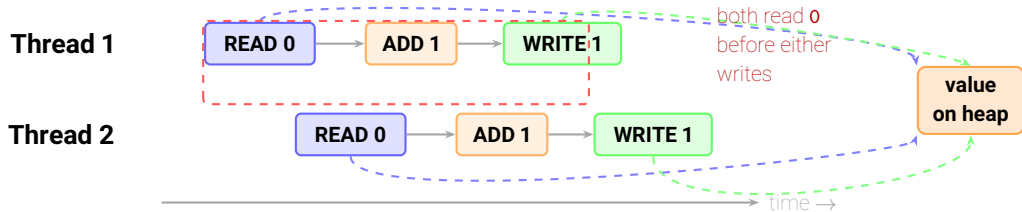
```
1  public class Counter {
2      private int value = 0;
3
4      public void increment() { value++; }    // NOT atomic!
5      public int get() { return value; }
6  }
7
8  Counter c = new Counter();
```

```
1  Thread t1 = new Thread(() -> {
2      for (int i = 0; i < 100_000; i++)
3          c.increment();
4  });
5  t1.start();
```

```
1  Thread t2 = new Thread(() -> {
2      for (int i = 0; i < 100_000; i++)
3          c.increment();
4  });
5  t2.start();
```

Why `value++` Loses Updates

`value++` **compiles to three steps -- threads can interleave between them.**



One increment lost in this interleaving

Both threads compute $0 + 1 = 1$ and write 1. Expected 2, got 1. Over 200,000 increments, many updates are silently dropped.

What Do You Think This Prints?

Two threads, one shared **String**

Vote: what does this print?

- Always **cat**?
- Always **dog**?
- Could be either?

```
1  String pet = null;    // shared on the
    heap
2
3  Thread t1 = new Thread(() -> {
4      if (pet == null) pet = "cat";
5  });
6  Thread t2 = new Thread(() -> {
7      if (pet == null) pet = "dog";
8  });
9
10 t1.start(); t2.start();
11 t1.join();  t2.join();
12
13 System.out.println(pet);
```

What Do You Think This Prints?

Two threads, one shared **String**

```
1  String pet = null;    // shared on the
    heap
2
3  Thread t1 = new Thread(() -> {
4      if (pet == null) pet = "cat";
5  });
6  Thread t2 = new Thread(() -> {
7      if (pet == null) pet = "dog";
8  });
9
10 t1.start(); t2.start();
11 t1.join();  t2.join();
12
13 System.out.println(pet);
```

Vote: what does this print?

- Always **cat**?
- Always **dog**?
- Could be either?

It is non-deterministic

Both threads can pass **pet == null** before either writes -- then the last write wins. You may get **cat** or **dog**. Same pattern as the counter: **check-then-act** on shared state without synchronization.

Interactive Break #1 -- Thread Bugs

Now that you know `value++` is unsafe -- find all three bugs

```
1  Counter c = new Counter();    // same class, value++ inside
2  Thread t1 = new Thread() -> {
3      for (int i = 0; i < 1000; i++) c.increment();
4  };
5  Thread t2 = new Thread() -> {
6      for (int i = 0; i < 1000; i++) c.increment();
7  };
8  t1.run()
9  t2.run();
10 System.out.println(c.get()); // expected: 2000 -- but what happens?
11 try {
12     t1.join();
13 } catch (InterruptedException e) {
14     // do nothing
15 }
```

Interactive Break #1 -- Answers

1. **counter[0]++ is not thread-safe.**

Increment is three operations (read -- add -- write). Two threads interleaving produce a final count less than 2000. Fix: **synchronized** block or **AtomicInteger**.

2. **t1.run() instead of t1.start().**

run() executes on the **current** thread sequentially -- no new thread is created. Both loops run on the main thread one after the other, so **join()** later has nothing to wait for (thread is already TERMINATED before the join).

3. **InterruptedException is silently swallowed.**

The interrupt signal is consumed but not restored. Any caller expecting the thread to honour cancellation will never see it. Fix: add **Thread.currentThread().interrupt();** inside the **catch** block.

synchronized -- The Simplest Fix

Option A: synchronized method

```
1  public class Counter {
2      private int value = 0;
3
4      public synchronized void
        increment() {
5          value++;    // only one
                        thread at a time
6      }
7      public synchronized int get() {
8          return value;
9      }
10 }
```

Option B: synchronized block (finer grained)

```
1  private final Object lock = new
    Object();
2
3  public void increment() {
4      synchronized (lock) {
5          value++;
6      }
7      // other non-critical work can
        run here
8  }
```

synchronized -- How It Works

Monitor locks on every object

Every Java object has a built-in **monitor lock** -- yes, literally every object, including a **new Object()** created purely to serve as one. **synchronized** guarantees:

- **Mutual exclusion:** only one thread executes the block at a time
- **Visibility:** writes are flushed to main memory; reads see the latest value

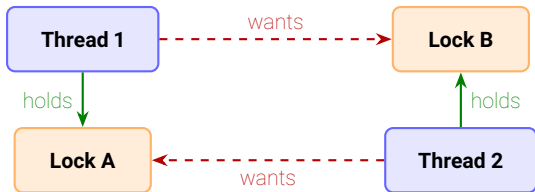
*Tip: you can also use **synchronized(this)** to lock on the current instance.*

Cost

Synchronisation *serialises* access -- threads queue up and wait. Use it only for the **minimum** critical section. Locking the whole method when only one line needs protection slows everything down.

Deadlock -- What to Watch For

Deadlock: two threads each hold a lock the other needs.



Both threads wait forever. Neither can proceed.

Classic analogy: two drivers meeting head-on on a one-lane bridge, each waiting for the other to reverse.

Rules to prevent deadlock

1. **Always acquire locks in the same fixed order.**

If every thread locks A before B, the circular wait cannot form.

2. **Minimise time inside a lock.**

Don't call external code (network, DB, UI) while holding a lock.

3. **Prefer one lock per object.**

The more locks involved, the harder reasoning becomes.

Detecting Deadlocks in IntelliJ

What to look for in the debugger

Run your app → click the **Threads** tab in the Debug view. A deadlock shows as two or more threads stuck in **BLOCKED** state, each waiting for a lock held by the other. The JVM hangs and never prints another line.

volatile -- Visibility Guarantee

Stop flag visible across all threads

```
1  // Flag visible across all threads:
2  private volatile boolean running = true
   ;
3
4  public void stop() {
5      running = false;    // write is
                           visible immediately
6  }
7
8  public void run() {
9      while (running) {    // always reads
                           fresh value
10         doWork();
11     }
12 }
```

Visibility -- not atomicity

Every thread sees the latest write of a **volatile** field. Perfect for a simple stop flag.

But **running = !running** is still a read-modify-write race. When you need both visibility *and* atomicity, use **Atomic***.

AtomicInteger -- Visibility and Atomicity

```
1  import java.util.concurrent.atomic.*;
2  AtomicInteger counter = new
    AtomicInteger(0);
3
4  // Thread-safe, no synchronized needed:
5  int before = counter.getAndIncrement();
    // read, then +1
6  int after  = counter.incrementAndGet();
    // +1, then read
7  counter.addAndGet(5);
    // add 5
8  int snapshot = counter.get();
    // read current value
9  // Advanced: compareAndSet() for lock-
    free logic
```

Drop-in replacement for `int` in shared state.

Also available:

-- `AtomicBoolean`

-- `AtomicLong`

-- `AtomicReference<T>`

*Use when a single shared variable needs atomic read-modify-write -- e.g. the **Counter** from earlier.*

Interactive Break #2 -- Spot the Race Condition

Two threads share a plain **ArrayList**

```
1  List<String> names = new ArrayList<>();
2
3  Thread t1 = new Thread(() -> {
4      for (int i = 0; i < 500; i++)
5          names.add("Alice-" + i);
6  });
7  Thread t2 = new Thread(() -> {
8      for (int i = 0; i < 500; i++)
9          names.add("Bob-" + i);
10 });
11
12 t1.start(); t2.start();
13 t1.join(); t2.join();
14
15 System.out.println(names.size()); // ???
```

1. What can actually happen when two threads call **add()** simultaneously?
2. What is the minimal fix?
3. Is **synchronized** always the right answer here? Can you name two alternatives?

Interactive Break #2 -- Answers (1/2)

1. What can happen:

`ArrayList` is not thread-safe. Two threads may resize the internal array simultaneously, overwrite each other's entries, or produce a corrupt internal state (`ConcurrentModificationException` or silent data loss). The size can be anything from 500 to 1000.

2. Minimal fix -- synchronized block:

```
1 // synchronize every add call on the same lock:
2 synchronized (names) { names.add("..."); }
```

Interactive Break #2 -- Answers (2/2)

3. Two better alternatives:

```
1 // Option A: Collections.synchronizedList
2 List<String> names =
3     Collections.synchronizedList(new ArrayList<>());
4
5 // Option B: CopyOnWriteArrayList (best for read-heavy cases)
6 List<String> names = new CopyOnWriteArrayList<>();
```

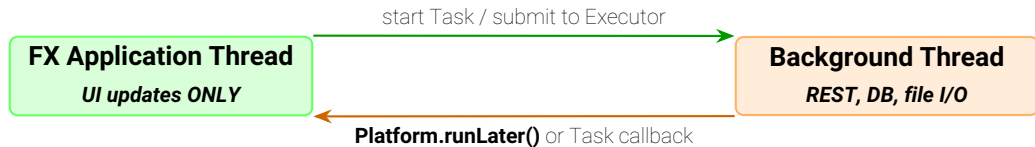
CopyOnWriteArrayList creates a new copy of the array on every write -- reads are always lock-free. Use it when reads vastly outnumber writes (e.g. a list of event listeners).

The Golden Rule of JavaFX Threading

Only the FX Application Thread may read or write UI components. Period.

Modifying a **Label**, **TableView**, or any other JavaFX node from a background thread throws **IllegalStateException: Not on FX application thread** at runtime -- or, worse, produces silent visual glitches that are hard to reproduce.

Sound familiar? If your JavaFX app ever threw **IllegalStateException: Not on FX application thread** and you had no idea why -- this is the reason. Now you can fix it.



Consequence: All heavy work (REST calls, database queries, file loading) must run off the FX thread. The result is then handed *back* to the FX thread to update the UI.

Platform.runLater() -- Simple Updates

Update a label from a background thread

```
1  // import javafx.application.Platform;
2
3  new Thread(() -> {
4      String data = fetchDataFromServer
        ();
5
6      Platform.runLater(() ->
7          resultLabel.setText(data)
8      );
9  }).start();
```

From any DAO or service class

```
1  // Called from a background thread:
2  public void onScoreLoaded(Score s) {
3      Platform.runLater(() -> {
4          nameLabel.setText(s.player);
5          scoreLabel.setText("" + s.
6              points);
7      });
8  }
```

Platform.runLater() -- When to Use It

When to use

- Short, fire-and-forget UI updates
- You already have the result in hand
- No progress reporting needed

Limitations

- No return value
- No built-in progress tracking
- Exceptions thrown inside **runLater** are uncaught unless you wrap them yourself
- For anything more complex, use **Task<T>**

Task<T> -- What Is It?

JavaFX's built-in answer to "do slow work, then update the UI"

`javafx.concurrent.Task<T>` wraps a background job that returns a result of type **T**. JavaFX handles the thread hand-off for you.

Compared to `Platform.runLater()`:

- **runLater** -- one-shot UI update when you already have the result
- **Task<T>** -- blocking work in `call()`, plus progress, success, failure, and cancellation

*Think of it as a **Runnable** with a return value and JavaFX-aware lifecycle callbacks.*

Task<T> -- Core Contract

Task<T> core contract

- Override `call()` -- runs on a **background** thread; may block freely
- `setOnSucceeded()` -- result ready; callback fires on **FX thread**
- `setOnFailed()` -- exception via `getException()`; fires on **FX thread**
- `setOnCancelled()` -- task was cancelled before finishing

The key insight

`call()` and the callbacks run on **different** threads -- the framework handles the hand-off.
You never call `Platform.runLater()` yourself when using **Task**.

Task<T> -- Background Fetch Example

Define the task

```
1 Task<List<Score>> loadTask = new
    Task<>() {
2     @Override
3     protected List<Score> call()
4         throws Exception {
5         return scoreDao.findAll();
6     }
7 };
```

Wire callbacks and start

```
1 loadTask.setOnSucceeded(e ->
2     tableView.setItems(
3         FXCollections.
4             observableArrayList(
5                 loadTask.getValue())));
6 loadTask.setOnFailed(e ->
7     errorLabel.setText(
8         loadTask.getException().
9             getMessage()));
10 new Thread(loadTask).start();
```

*Callbacks fire on the FX thread automatically -- safe to update **tableView** and **errorLabel**.*

Task<T> -- Reporting Progress

Update progress from `call()`

Live-updating properties

```
1    Task<List<Score>> loadTask = new Task
      <>() {
2        @Override
3        protected List<Score> call()
4            throws IOException,
              InterruptedException {
5            updateMessage("Loading scores...
              ");
6            updateProgress(0, 1);
7            List<Score> result = scoreDao.
              findAll();
8            updateProgress(1, 1);
9            return result;
10        }
11    };
```

- `updateMessage(String)` -- sets `messageProperty()`
- `updateProgress(done, total)` -- sets `progressProperty()` (0.0 -- 1.0)
- Both are thread-safe: call from `call()` on the bg thread; JavaFX propagates to the FX thread automatically

Task<T> -- Binding to the UI

Bind UI controls to the task's properties

```
1 progressBar.progressProperty()  
2     .bind(loadTask.progressProperty());  
3 statusLabel.textProperty()  
4     .bind(loadTask.messageProperty());
```

JavaFX property binding

Binding links two properties so one mirrors the other. When

loadTask.progressProperty() changes, **progressBar** updates instantly -- no event handler needed.

Daemon Threads

Background helpers vs. keep-alive threads

Daemon (OS term) = a background helper the JVM does *not* wait for when shutting down.

If a **non-daemon** thread is still running when the user closes the JavaFX window, the JVM stays alive -- the process hangs in the background.

Mark background workers as daemon: **t.setDaemon(true)** so the app exits cleanly.

Rule of thumb for JavaFX

Main and FX Application Thread are non-daemon. REST loaders, file I/O, and **Task** workers should run on daemon threads.

Background Work -- Thread vs. Executor

Option A: `new Thread(...).start()`

Create one OS thread per job. Fine for learning and occasional JavaFX tasks.

```
1  Runnable job = () -> { /* slow work
    */ };
2  Thread t = new Thread(job);
3  t.setDaemon(true);
4  t.start();
```

Option B: `ExecutorService`

Submit jobs to a thread pool -- reuse threads, cap concurrency. Better when you spawn many tasks.

```
1  ExecutorService exec =
2      Executors.newCachedThreadPool(r
    -> {
3      Thread t = new Thread(r);
4      t.setDaemon(true);
5      return t;
6  });
7  exec.submit(job);
```

*Thread pools, **Future**, and shutdown -- covered in detail in the next section.*

Virtual Threads (Java 21+)

Option C: lightweight threads for blocking I/O

Virtual threads are cheap to create -- the JVM maps many of them onto few OS threads. Ideal for many parallel REST or DB calls.

```
1  Thread.startVirtualThread(() -> {  
2      /* blocking I/O -- OK */  
3  });
```

When to choose which

- **Learning / JavaFX:** plain `Thread` or `ExecutorService`
- **Many parallel blocking calls:** virtual threads (Java 21+)
- **Production JavaFX:** often `ExecutorService` with a daemon thread factory

Running a Task<T>

Apply the same three patterns to a JavaFX **Task**:

Plain Thread

```
1 Task<List<Score>>  
    task = ...;  
2 Thread t = new  
    Thread(task);  
3 t.setDaemon(true);  
4 t.start();
```

ExecutorService

```
1 executor.submit(task);  
2 // pool created once  
    with  
3 // daemon thread factory
```

Virtual thread

```
1 Thread.  
    startVirtualThread  
    (task);
```

For this course, `new Thread(task).start()` is enough. Prefer an executor when you run many tasks in parallel.

Thread Pools with `ExecutorService`

Creating and using a thread pool

```
1  ExecutorService executor =
2      Executors.newFixedThreadPool(4);
3
4  Future<List<Score>> future =
5      executor.submit(() -> scoreDao.
6          findAll());
7
8  List<Score> scores = future.get(); //
9      may throw
10
11  executor.shutdown();
12
13 // Java 19+: try-with-resources also
14     works
```

`Callable` vs `Runnable`

`Runnable.run()` returns `void`.

`Callable.call()` returns a value and can throw checked exceptions.

Use `Callable` (with `submit()`) when you need a result back.

ExecutorService -- Pool Types and Pitfalls

Three common pool types

<code>newFixedThreadPool(n)</code>	exactly <i>n</i> threads
<code>newCachedThreadPool()</code>	grows/shrinks as needed
<code>newSingleThreadExecutor()</code>	one thread, serial queue

Why use a pool?

- Thread creation is expensive (OS resources, stack allocation)
- A pool creates threads once and reuses them
- Caps the number of concurrent threads -- prevents resource exhaustion when many tasks arrive at once

future.get() blocks the calling thread

Don't call **future.get()** on the FX Application Thread -- it will freeze the UI just like a direct REST call. Use **Task<T>** or **CompletableFuture** with callbacks instead.

CompletableFuture

Master **Task<T>** first

If you use **Task<T>** for this course, you are fully covered. **CompletableFuture** shines in production chains -- study it when you outgrow **Task**.

Key methods

supplyAsync	start async computation
thenApply	transform result
thenAccept	consume result (no return)
thenAcceptAsync	consume on given executor
exceptionally	handle errors in the chain
thenCombine	merge two futures

CompletableFuture -- Async Pipeline

Async load + JavaFX update in a pipeline

```
1    CompletableFuture
2        .supplyAsync(() -> scoreDao.findAll())
3
4        .thenApply(scores ->
5            FXCollections.observableArrayList(scores))
6
7        .thenAcceptAsync(
8            list -> tableView.setItems(list),
9            Platform::runLater)    // executor = runLater!
10
11        .exceptionally(ex -> {
12            Platform.runLater(() ->
13                errorLabel.setText(ex.getMessage()));
14            return null;
15        });
```

CompletableFuture -- Connection to Session 11

From `HttpClient.sendAsync()` to the UI

`HttpClient.sendAsync()` returns a `CompletableFuture<HttpResponse>`. You can chain JSON parsing and the UI update directly -- no manual `Platform.runLater()` needed if you pass it as the executor to `thenAcceptAsync`.

Pattern: network call → parse → `thenAcceptAsync(..., Platform::runLater)` → update UI.

Choosing the Right Tool

Need	Tool	Why
Quick UI update	<code>Platform.runLater()</code>	fire-and-forget
JavaFX task + progress	<code>Task<T></code>	FX callbacks
Many tasks efficiently	<code>ExecutorService</code>	thread pool
Chain async steps	<code>CompletableFuture</code>	pipeline
Thread-safe counter	<code>AtomicInteger</code>	lock-free
Rarely-written list	<code>CopyOnWriteArrayList</code>	lock-free reads
Often-written list	<code>Collections.synchronizedList</code>	simple wrapper
Java 21+ bg work	Virtual Threads	cheap, JVM-managed

Choosing the Right Tool -- Rules of Thumb

Rule of thumb for this course

JavaFX + REST call: `Task<T>` is usually the cleanest choice -- it handles progress, success, and failure with FX-thread-safe callbacks out of the box.

For production systems

CompletableFuture scales better when you have chains of async operations or need to combine results from multiple sources (e.g. two parallel API calls merged into one response).

Patterns You Know -- Now Multithreaded

Command + ExecutorService

An **ExecutorService** is a natural **Command queue**. Submit **Callable** or **Runnable** objects -- the pool executes them asynchronously, in order or in parallel.

```
1  executor.submit(new SaveScoreCommand(  
    score));  
2  executor.submit(new SendEmailCommand(  
    player));  
3  // both run in the pool concurrently
```

Observer + threading

JavaFX property listeners (**ChangeListener**) always fire on the thread that changes the property. If you change a property from a background thread, listeners fire on that thread -- and may try to touch the UI.

Solution: update properties only on the FX thread (inside **Task.setOnSucceeded()** or **Platform.runLater()**).

What We Covered Today

Concurrency Foundations

- **Thread** / **Runnable**: create and start threads
- Lifecycle: NEW → RUNNABLE → BLOCKED → TERMINATED
- **sleep()**, **join()**, daemon threads

Shared State & Safety

- Race conditions on shared state
- **synchronized**: mutual exclusion + visibility
- Deadlock: consistent lock order;
AtomicInteger, **volatile**

JavaFX Threading

- Only the FX Application Thread touches UI
- **Platform.runLater()**: fire-and-forget updates
- **Task<T>**: progress, success, failure on FX thread

Executors & Futures

- **ExecutorService**: thread pool, cap concurrency
- **CompletableFuture**: async pipeline + **Platform::runLater**
- Virtual Threads (Java 21): lightweight, JVM-managed